🏃 RunRoute AI

AI-Powered Running Route Recommendation App

Product Requirements Document v2.0

검증 완료 + 개발 아키텍처 포함 | 2025 년 3 월

항목	내용
문서명	RunRoute AI PRD v2.0
이전 버전	v1.0 → v2.0 (PRD 검증 + 아키텍처 추가)
플랫폼	iOS / Android / WearOS / watchOS
상태	검증 완료 (Validated)
주요 변경	PRD 검증 섹션, 상세 개발 아키텍처, API 설계, DB 스키마, 배포 전략 추가

0. PRD 검증 결과 (Validation Report)

v1.0 PRD 를 7 개 검증 기준으로 평가한 결과입니다. 각 항목의 보완 사항을 v2.0 에 반영하였습니다.

0.1 종합 검증 스코어카드

검증 항목	v1.0 상태	판정	v2.0 보완 내용
문제 정의 명확성	사용자 페인포인트 4 가지 정의됨, 시장 규모 데이터 없음	⚠️ 보통	글로벌 러닝 앱 시장 규모 및 TAM/SAM/SOM 추가
사용자 스토리 완성도	페르소나 4 개 정의, User Story 형식 미비	⚠️ 보통	각 페르소나별 User Story + Acceptance Criteria 추가
기능 요구사항 상세화	기능 목록 존재, 각 기능 상세 스펙 없음	⚠️ 보통	기능별 입력/출력/예외처리 스펙 추가
기술 아키텍처 구체성	컴포넌트 나열 수준, 실제 설계 없음	❌ 미흡	전체 시스템 아키텍처, API 설계, DB 스키마 신규 추가
비기능 요구사항	주요 항목 정의됨, 테스트 기준 없음	⚠️ 보통	각 NFR 에 대한 테스트 방법 및 측정 기준 추가
리스크 관리	리스크 목록 있음, 대응 계획 구체성 부족	⚠️ 보통	리스크별 대응 시나리오 및 담당자 책임 추가
출시 전략 (GTM)	로드맵만 존재, GTM 전략 없음	❌ 미흡	Go-to-Market 전략, 마케팅 채널, 론칭 체크리스트 추가

0.2 PRD 검증 세부 분석

[보완] 시장 규모 (TAM / SAM / SOM)

구분	정의	규모 (2024)	근거
TAM	글로벌 러닝/피트니스 앱 시장 전체	$15.2B	Grand View Research 2024
SAM	스마트워치 연동 러닝 앱 시장	$3.1B	GPS + 워치 결합 세그먼트
SOM	AI 루트 추천 특화 틈새시장	$180M	출시 5 년 내 목표 점유 1%

[보완] User Story + Acceptance Criteria (핵심 기능)

기능	User Story	Acceptance Criteria
루트 추천	여행 중인 러너로서, 현재 위치 기반으로 원하는 거리의 루트를 3 초 이내에 받고 싶다	1) GPS 획득 < 5 초 2) 루트 3 개 생성 < 3 초 3) 거리 오차 ±10% 4) 오프라인 시 안내 메시지
워치 단독 실행	폰 없이 워치만으로 러닝을 시작하고 싶다	1) 워치 앱 콜드 스타트 < 3 초 2) LTE/WiFi 없어도 마지막 캐시 루트 사용 3) 배터리 1 시간 사용 시 ≤15%
자연어 입력	복잡한 설정 없이 말하듯 원하는 루트 조건을 입력하고 싶다	1) "5km 공원" 입력 시 의도 파싱 정확도 ≥ 90% 2) 인식 불가 시 슬라이더 UI 로 폴백 3) 응답 < 2 초
턴-바이-턴 내비	뛰면서 화면 안 보고도 방향을 알고 싶다	1) 회전 지점 50m 전 햅틱 2) 루트 이탈 시 10 초 이내 재탐색 3) 음성 안내 지원

1. 개요 (Executive Summary)

RunRoute AI 는 새로운 장소에서 러너가 원하는 거리와 조건을 자연어로 입력하면 AI 가 최적 러닝 루트를 즉시 추천하는 멀티플랫폼 앱입니다. Apple Watch·WearOS 단독 실행, iOS·Android 앱을 통해 어디서나 안전하고 즐거운 러닝 경험을 제공합니다.

핵심 가치	설명
즉시성	새 도시에서 앱 실행 3 초 내 루트 추천 완료
자연어 UX	"강변으로 5km 가볍게" → AI 가 구조화된 파라미터로 자동 변환
워치 퍼스트	스마트워치 단독 실행 – 폰 없이 손목에서 모든 것 해결
안전 최우선	야간 조명, 인도 여부, 치안 등급 반영한 안전 루트 제공

2. 문제 정의

#	문제	현재 해결 방법	불편함
1	새로운 장소에서 어디를 뛰어야 할지 모름	Google Maps 수동 검색	시간 소요, 러닝 최적화 없음
2	원하는 거리에 맞는 루트 찾기 어려움	직접 지도 측정	번거롭고 부정확
3	워치에서 바로 사용 불가	폰으로 설정 후 워치 연동	설정 복잡
4	안전한 경로인지 알 수 없음	인터넷 검색, 현지인 문의	정보 부족

3. 목표 및 성공 지표

KPI	목표값	측정 방법	달성 시점
루트 추천 응답 시간	< 3 초	서버 P95 응답 모니터링	MVP 출시
루트 만족도	4.2 점/5 점	세션 종료 후 인앱 평가	출시 1 개월
Day-7 리텐션	40% 이상	코호트 분석	출시 2 개월
워치 단독 실행 성공률	95% 이상	워치 세션 로그	MVP 출시
MAU	50,000 명	Analytics 대시보드	출시 6 개월

KPI	목표값	측정 방법	달성 시점
Pro 전환율	15% 이상	결제 퍼널 분석	출시 6 개월

4. 기능 요구사항

4.1 AI 루트 추천 엔진 (F-01)

항목	상세
입력	현재 위치(GPS), 목표 거리(km), 지형 선호(공원/강변/도심/언덕), 난이도, 자연어 텍스트
처리	자연어 파싱 → 파라미터 추출 → 후보 루트 생성(OSRM) → AI 스코어링 → Top 3 선택
출력	루트 3 가지 (GeoJSON), 예상 시간, 고도 그래프, 안전도 점수, 경관 포인트
예외 처리	GPS 미획득: 수동 위치 입력 | 루트 없음: 반경 확대 재시도 | API 실패: 캐시 루트 제공
성능 기준	P95 < 3 초, P99 < 5 초, 가용성 99.9%

4.2 스마트워치 앱 (F-02)

기능	Apple Watch	WearOS
독립 실행	Standalone (watchOS 9+, LTE)	Standalone (WearOS 3+, LTE)
음성 입력	Siri Shortcut 연동	Google Assistant / on-device STT
내비게이션	턴-바이-턴, Haptic 피드백	턴-바이-턴, Vibration 패턴
센서	심박수, GPS, 가속도계	심박수, GPS, 가속도계
화면 구성	루트맵 / 페이스 / 거리 / 심박 4 화면	루트맵 / 페이스 / 거리 / 심박 4 화면
배터리 목표	1 시간 러닝 ≤ 15%	1 시간 러닝 ≤ 15%

4.3 기능 우선순위 매트릭스

기능	중요도	난이도	우선순위	릴리즈
AI 루트 추천	높음	높음	P0	MVP
모바일 내비게이션	높음	중간	P0	MVP
스마트워치 앱	높음	높음	P0	MVP
자연어 입력	높음	높음	P1	MVP
러닝 기록/통계	중간	낮음	P1	MVP
오프라인 루트 저장	중간	중간	P2	v1.1

기능	중요도	난이도	우선순위	릴리즈
소셜/공유	낮음	중간	P2	v1.2
AR 내비게이션	중간	매우 높음	P3	v2.0

5. 개발 아키텍처 (Development Architecture)

5.1 전체 시스템 아키텍처 개요

RunRoute AI 는 클라이언트(모바일/워치) – API Gateway – 마이크로서비스 – AI 엔진 – 외부 서비스로 구성된 클라우드 네이티브 아키텍처를 채택합니다.

[클라이언트 레이어]
iOS App | Android App | Apple Watch | WearOS

▼ *HTTPS / WebSocket* ▼

[API Gateway (AWS API Gateway + CloudFront)]
인증(JWT) | 레이트 리미팅 | 로드 밸런싱 | SSL 종단

▼ *내부 REST API* ▼

[마이크로서비스 레이어 (NestJS / Docker / ECS)]
Auth Service | Route Service | User Service | Notification Service | Analytics Service

▼ *Python gRPC* ▼

[AI 엔진 레이어 (Python FastAPI / AWS SageMaker)]
NLP 파서 (LangChain + GPT-4o) | 루트 스코어링 (ML) | 안전도 분석 | 추천 엔진

▼ *외부 API 호출* ▼

[외부 서비스]
Google Maps API | OSRM (루트 최적화) | OpenWeather API | OpenAI API | Firebase (푸시)

▼ *데이터 레이어* ▼

[데이터 레이어]
PostgreSQL (AWS RDS) | Redis (ElastiCache) | S3 (루트 데이터) | Elasticsearch (검색)

5.2 마이크로서비스 상세

서비스명	역할	기술 스택	주요 API
Auth Service	회원가입/로그인, JWT 발급, 소셜 로그인	NestJS, Passport.js, bcrypt	POST /auth/login, POST /auth/refresh, POST /auth/social
Route Service	루트 생성, 추천, 저장, 조회	NestJS, TypeORM, Redis, OSRM	POST /routes/recommend, GET /routes/:id, POST /routes/save
User Service	프로필, 선호 설정, 러닝 기록 관리	NestJS, TypeORM, S3	GET /users/me, PATCH /users/preferences, GET /users/history
AI Service	자연어 파싱, 루트	Python, FastAPI,	POST /ai/parse, POST /ai/score,

서비스명	역할	기술 스택	주요 API
	스코어링	LangChain, scikit-learn	POST /ai/recommend
Notification Service	푸시 알림, 이메일, 인앱 알림	NestJS, Firebase Admin, SES	POST /notify/push, POST /notify/email
Analytics Service	이벤트 수집, 대시보드 데이터	NestJS, Kafka, ClickHouse	POST /events, GET /analytics/dashboard

5.3 클라이언트 아키텍처

iOS 앱 아키텍처

레이어	구성 요소	역할
Presentation	SwiftUI Views, ViewModels (MVVM)	화면 렌더링, 상태 관리
Domain	Use Cases, Entities, Repository Protocols	비즈니스 로직, 도메인 모델
Data	API Client (URLSession), CoreData, UserDefaults	네트워크 통신, 로컬 저장
Framework	MapKit, CoreLocation, HealthKit, WatchConnectivity	OS 기능 연동
Watch Bridge	WCSession (양방향 데이터 동기화)	iPhone ↔ Apple Watch 연동

Android 앱 아키텍처

레이어	구성 요소	역할
UI	Jetpack Compose, ViewModel (MVVM)	선언형 UI, 상태 관리
Domain	Use Cases, Repository Interfaces	비즈니스 로직
Data	Retrofit2 (API), Room DB, DataStore	네트워크, 로컬 DB, 설정
DI	Hilt (의존성 주입)	모듈 간 결합도 최소화
Wear Bridge	Wearable Data Layer API, MessageClient	Android ↔ WearOS 연동

스마트워치 공통 아키텍처

컴포넌트	역할	비고
Route Cache Manager	루트 데이터 로컬 캐시 (최근 10 개)	폰 없이도 사용 가능
GPS Manager	위치 추적 최적화 (1 초 간격, 배터리 절약 모드)	백그라운드 유지
Navigation Engine	턴-바이-턴 계산, 이탈 감지, 재탐색	On-device 처리
Haptic Controller	방향별 진동 패턴 (좌: 2 회, 우: 1 회, 도착: 3 회)	화면 없이 안내
Health Monitor	심박수 실시간 수집, HealthKit/Google Fit 저장	운동 데이터 동기화

5.4 API 설계

핵심 API 엔드포인트

Method	Endpoint	Request Body (주요)	Response	인증
POST	/api/v1/routes/recommend	{ lat, lng, distance_km, preferences: [], natural_query }	{ routes: [3 개], metadata }	JWT
GET	/api/v1/routes/:id	-	{ route, elevation, safety_score, pois }	JWT
POST	/api/v1/routes/:id/start	{ device_type: 'watch'|'mobile' }	{ session_id, navigation_data }	JWT
PATCH	/api/v1/routes/:id/complete	{ actual_distance, duration, avg_pace }	{ record_id, stats }	JWT
POST	/api/v1/ai/parse	{ natural_query, context: { lat, lng } }	{ distance, terrain, difficulty, tags }	API Key
GET	/api/v1/users/me/history	?page=1&limit=20	{ records: [], total }	JWT
POST	/api/v1/auth/social	{ provider, token }	{ access_token, refresh_token, user }	-
GET	/api/v1/routes/nearby	?lat=&lng=&radius_km=5	{ cached_routes: [] }	JWT

WebSocket 이벤트 (실시간 내비게이션)

이벤트	방향	데이터	설명
navigation:update	서버 → 클라이언트	{ current_pos, next_turn, distance_to_turn, pace }	1 초마다 내비 데이터 전송
navigation:off_route	서버 → 클라이언트	{ deviation_meters, recalculating: true }	이탈 감지 시 즉시 전송
navigation:rerouted	서버 → 클라이언트	{ new_route, eta }	재탐색 완료 후 전송
location:update	클라이언트 → 서버	{ lat, lng, accuracy, timestamp }	1 초마다 위치 전송
session:complete	클라이언트 → 서버	{ session_id, distance, duration }	러닝 완료 시 전송

5.5 데이터베이스 스키마

주요 테이블 구조 (PostgreSQL)

테이블명	주요 컬럼	설명
users	id(UUID), email, social_provider, preferences(JSONB), created_at	사용자 기본 정보
routes	id(UUID), creator_id, geojson(JSONB), distance_km, elevation_data(JSONB), safety_score, terrain_tags(TEXT[])	루트 데이터 (GeoJSON 저장)
running_records	id(UUID), user_id, route_id, started_at, completed_at, actual_distance, avg_pace, heart_rate_avg, calories	러닝 기록
route_ratings	id(UUID), user_id, route_id, rating(1-5), feedback_tags(TEXT[]), comment, created_at	루트 평가
sessions	id(UUID), user_id, route_id, device_type, status, ws_channel_id, created_at	활성 내비게이션 세션
cached_routes	id(UUID), geohash(VARCHAR), routes(JSONB), expires_at	지역별 루트 캐시 (Redis 미스 시 폴백)

Redis 캐시 전략

캐시 키 패턴	TTL	데이터	목적
route:recommend:{geohash6}:{distance}	24 시간	추천 루트 3 개 (JSON)	동일 지역 반복 요청 응답 단축
user:session:{user_id}	2 시간	현재 러닝 세션 데이터	빠른 세션 상태 조회
user:preferences:{user_id}	1 시간	사용자 선호 설정	AI 파라미터 빠른 조회
route:safety:{route_id}	6 시간	안전도 분석 결과	안전도 재계산 방지
ai:parse:{query_hash}	30 분	자연어 파싱 결과	동일 쿼리 AI 호출 절감

5.6 AI 루트 추천 엔진 상세

처리 파이프라인

단계	처리 내용	기술	소요 시간
1. NLP 파싱	자연어 → {distance, terrain, difficulty, tags} 구조화	LangChain + GPT-4o (few-shot)	< 800ms
2. 위치 분석	현재 위치 Geohash 변환, 주변	PostGIS, Google Places	< 200ms

단계	처리 내용	기술	소요 시간
	POI 조회	API	
3. 후보 생성	반경 내 출발점 후보 → 그래프 탐색 → 루프 루트 생성	OSRM + A* 알고리즘	< 500ms
4. 스코어링	안전도/경관/고도/개인화 가중치 적용 스코어 계산	scikit-learn 랜덤포레스트	< 300ms
5. 후처리	Top 3 선택, GeoJSON 변환, 경관 포인트 마킹	Python/Shapely	< 200ms
전체 P95	모든 단계 합산	-	< 2.5 초

루트 스코어링 가중치

스코어 항목	가중치	데이터 소스	계산 방법
안전도	30%	범죄 공공데이터, 가로등 DB, 인도 여부	0-100 정규화
경관 점수	25%	Google Places 평점, 사진 수, 방문자 리뷰	POI 밀도 + 평점 가중
고도 쾌적도	20%	SRTM 고도 데이터	목표 난이도 대비 편차 최소화
거리 정확도	15%	OSRM 실제 거리 계산	목표 거리 대비 오차율
개인화	10%	사용자 과거 러닝 기록, 평가 데이터	협업 필터링 유사도

5.7 인프라 및 배포 아키텍처

AWS 인프라 구성

AWS 서비스	용도	구성	비용 최적화
ECS Fargate	마이크로서비스 컨테이너 실행	Auto Scaling (최소 2, 최대 20 태스크)	Spot 인스턴스 70% 활용
RDS PostgreSQL	메인 데이터베이스	Multi-AZ, db.t3.medium → db.r5.large (스케일업)	Reserved Instance 1 년
ElastiCache Redis	캐시 레이어	클러스터 모드, 3 개 샤드	데이터 압축으로 메모리 절감
SageMaker	AI 모델 배포	엔드포인트 Auto Scaling	Serverless Inference (저트래픽)
API Gateway	API 엔드포인트 관리	Regional, WebSocket 지원	캐싱 활성화로 Lambda 호출 감소
CloudFront	CDN + DDoS 방어	Edge Location 글로벌	Cache-Control 최적화
S3	루트 GeoJSON, 이미지 저장	Intelligent Tiering	오래된 데이터 Glacier 이동
MSK (Kafka)	이벤트 스트리밍	브로커 3 개, Replication Factor 3	메시지 압축 (Snappy)

CI/CD 파이프라인

단계	도구	트리거	소요 시간
코드 품질 검사	ESLint, Prettier, SonarQube	PR 생성 시	~2 분
단위 테스트	Jest (TS), pytest (Python), XCTest (iOS)	PR 생성 시	~5 분
통합 테스트	Supertest, Testcontainers	main 브랜치 머지 시	~10 분
컨테이너 빌드	Docker, ECR (AWS)	main 머지 후	~3 분
스테이징 배포	ECS Blue/Green Deploy	자동	~5 분
E2E 테스트	Detox (모바일), Cypress (API)	스테이징 배포 후	~15 분
프로덕션 배포	ECS Blue/Green, 카나리 (10% → 100%)	수동 승인 후	~10 분
모니터링 확인	CloudWatch, Datadog 알람	배포 완료 후 30 분	자동

환경 구성

환경	용도	데이터	배포 주기
Development	개발자 로컬 개발	Mock 데이터	즉시
Staging	QA 테스트, 스테이징 검증	프로덕션 익명화 데이터	PR 머지마다
Production	실서비스	실데이터	주 1-2 회 릴리즈

5.8 보안 아키텍처

보안 항목	적용 방법	기준
인증/인가	JWT (Access 15 분 + Refresh 30 일), OAuth 2.0 소셜 로그인	OWASP Top 10 준수
API 보안	API Gateway 레이트 리미팅 (100 req/min), WAF 룰셋	DDoS 방어
데이터 암호화	전송: TLS 1.3, 저장: AES-256 (RDS 암호화, S3 SSE)	GDPR / 개인정보보호법
위치 데이터	최소 수집 원칙, 러닝 세션 종료 후 raw GPS 삭제, 집계 데이터만 보관	개인정보보호법 제 16 조
취약점 관리	Snyk (의존성 취약점), SAST (SonarQube), 분기별 펜테스트	CVE 등급 High 이상 즉시 패치
시크릿 관리	AWS Secrets Manager, 환경변수 코드 노출 금지	GitGuardian 자동 스캔

5.9 모니터링 & 관찰성

항목	도구	알람 조건	담당
APM (성능 모니터링)	Datadog APM	P95 응답시간 > 3 초	백엔드팀
에러 추적	Sentry (모바일 + 서버)	에러율 > 1%	전체팀
인프라 모니터링	CloudWatch + Grafana	CPU > 80%, 메모리 > 85%	인프라팀
비즈니스 메트릭	Mixpanel + 자체 Analytics	DAU 전주 대비 20% 하락	PM 팀
AI 품질 모니터링	자체 대시보드	루트 만족도 < 3.5 점	AI 팀
보안 이벤트	AWS GuardDuty + CloudTrail	비정상 API 호출 패턴	보안팀

6. 플랫폼별 요구사항

항목	iOS	Android	Apple Watch	WearOS
최소 버전	iOS 16.0+	Android 9.0+	watchOS 9.0+	WearOS 3.0+
언어/프레임워크	Swift 5.9 / SwiftUI	Kotlin / Jetpack Compose	SwiftUI for Watch	Compose for Wear
지도	MapKit	Google Maps SDK	MKMapView (경량)	Maps for Wear
AI 연동	REST API + CoreML(경량)	REST API + TFLite(경량)	REST API (LTE)	REST API (LTE)
건강 연동	HealthKit	Health Connect	HealthKit (직접)	Health Services
오프라인	CoreData + URLCache	Room + OkHttp Cache	WatchKit Storage (5MB)	WearOS Storage
배포	App Store	Google Play	App Store (Watch)	Google Play (Wear)

7. 비기능 요구사항

분류	요구사항	목표	테스트 방법
성능	루트 추천 응답	P95 < 3 초	k6 부하 테스트 (1000 VU)
성능	앱 콜드 스타트	< 2 초	Firebase Performance 자동 측정
가용성	API 업타임	99.9% SLA	Pingdom 5 분 간격 체크
배터리	워치 1 시간 러닝	≤ 15% 소모	실기기 배터리 로그 분석
배터리	모바일 1 시간 러닝	≤ 10% 소모	Android Battery Historian
보안	위치 데이터 암호화	TLS 1.3 + AES-256	SSL Labs A+ 등급
접근성	시각 장애인 지원	WCAG 2.1 AA 등급	VoiceOver / TalkBack 수동 테스트
확장성	동시 접속자	10,000 CCU	Locust 스트레스 테스트

8. 개발 로드맵

단계	기간	주요 마일스톤	산출물
Phase 1: 설계	1-4 주차	UX 리서치, 시스템 설계, DB 스키마 확정, API 설계	설계 문서, ERD, API 스펙
Phase 2: 백엔드 MVP	5-10 주차	Auth/Route/User 서비스, AI 엔진 v1, REST API	백엔드 API Beta
Phase 3: 모바일 MVP	7-14 주차	iOS / Android 앱 핵심 기능	모바일 앱 Beta
Phase 4: 워치 앱	11-18 주차	Apple Watch / WearOS 앱 개발	워치 앱 Beta
Phase 5: QA & 최적화	17-21 주차	E2E 테스트, 성능 최적화, 보안 감사	QA 완료 빌드
Phase 6: 출시	22-24 주차	스토어 심사 제출, 소프트 론칭 (한국)	v1.0 Public
Phase 7: 성장	25 주차~	피드백 반영, Pro 기능, 글로벌 확장	v1.1, v1.2

9. 출시 전략 (Go-to-Market)

9.1 타겟 채널 및 전략

채널	전략	목표 지표	예산 비중
러닝 커뮤니티	마라톤 대회 스폰서, 러닝 클럽 파트너십	브랜드 인지도, 입소문	30%
소셜 미디어	인스타그램 러닝 인플루언서 협업, 릴스 콘텐츠	앱 다운로드	25%
앱스토어 ASO	키워드 최적화: "러닝 루트", "달리기 앱"	오가닉 다운로드	15%
여행 플랫폼	트리플, 여기어때 파트너십, 여행자 특화 노출	해외/여행 유저 유입	20%
PR/언론	IT 매체 리뷰, 스타트업 미디어 기고	신뢰도 확보	10%

9.2 론칭 체크리스트

분류	항목	담당	완료 기준
스토어	App Store 심사 제출 (7 일 여유)	iOS 팀	심사 통과
스토어	Google Play 심사 제출 (3 일 여유)	Android 팀	심사 통과

분류	항목	담당	완료 기준
인프라	프로덕션 환경 스트레스 테스트	인프라팀	10,000 CCU 통과
보안	외부 보안 감사 완료	보안팀	High 이상 취약점 0 건
법무	개인정보처리방침, 이용약관 법무 검토	법무팀	법무 승인
CS	고객지원 운영 준비 (FAQ, 채널톡)	운영팀	CS 채널 오픈
마케팅	론칭 프레스 키트, SNS 예약 발행	마케팅팀	콘텐츠 승인
모니터링	알람 및 대응 매뉴얼 준비	전체팀	On-call 로테이션 확정

10. 리스크 관리

리스크	가능성	영향도	완화 전략	담당
지도 API 비용 초과	높음	높음	OSM 병행, 호출 캐싱, 월 예산 알람 설정	인프라팀
AI 루트 품질 미흡	중간	높음	A/B 테스트, 사용자 피드백 루프, 모델 재학습 파이프라인	AI 팀
워치 배터리 소모 과다	중간	높음	GPS 샘플링 주기 동적 조정, 저전력 모드 구현	워치팀
앱스토어 심사 반려	낮음	중간	Apple/Google 가이드라인 사전 검토, 2 주 여유 일정	전체팀
개인정보 법규 이슈	낮음	높음	법무 검토, 최소 수집 원칙, DPO 지정	법무팀
OpenAI API 장애	낮음	높음	Fallback: Claude API, 자체 경량 모델 온디바이스 배포	AI 팀

11. 수익화 전략

플랜	가격	주요 기능	타겟
Free	무료	루트 추천 3 회/일, 기본 내비게이션, 7 일 기록	신규 유저
Pro Monthly	월 ₩5,900	무제한 추천, 오프라인, 고급 통계, AR 내비	주 3 회+ 러너

플랜	가격	주요 기능	타겟
Pro Annual	연 ₩49,900	Pro 전체 + 코치 AI + 우선 지원 (30% 할인)	헤비 유저
Team	월 ₩9,900/인 (5 인 +)	그룹 챌린지, 팀 통계, 관리자 대시보드	러닝 클럽/기업

RunRoute AI PRD v2.0 – End of Document